CSA17 Assignment 1 – Hospital Medicine-Delivery Robot

Task 1 – Agent and Algorithm Design

The robot is a goal-based, model-based rational agent; I compare three uninformed strategies: BFS (iterative, queue), Uniform-Cost Search (iterative, priority queue) and Iterative Deepening DFS (recursive).

1.1 PEAS description

Element	Specification
Performance measure	Deliver the medicine to the requested ward; minimise total travel time; no collisions; low computation and memory
Environment	Hospital: junctions, corridors, lifts, ward doors, modelled as a weighted graph
Actuators	Wheel motors (move along an edge), lift/door interface, medicine tray lock
Sensors	Localisation (position), obstacle/bump sensors, door/lift status, ward ID reader

1.2 Environment classification

Property	Choice	Justification
Observable	Fully observable (original design)	Localisation gives the exact location; the map is known. Day-4 failure makes it partially observable (Task 12)
Deterministic	Deterministic	Moving along an edge reaches the neighbouring node with a fixed travel-time cost
Episodic / sequential	Sequential	Each move changes the position and affects all later moves and the total cost
Static / dynamic	Static (for planning)	The map is assumed fixed while the route is computed; real traffic would make it semi-dynamic
Discrete / continuous	Discrete	Locations are graph nodes and moves are edges (the continuous motion is handled by low-level control)
Agents	Single-agent	Only the robot plans; people and other robots are treated as obstacles

1.3 View of AI and rationality

The robot follows the **acting rationally** view: it selects the action sequence that is expected to achieve the best outcome (reach the ward with least cost) given its percepts and knowledge, rather than imitating humans or reproducing human reasoning. It is a rational problem-solving agent because, for every percept sequence, it chooses the plan maximising its performance measure (a goal-based agent with a model of the map). Route finding on a graph is the classic early AI problem-solving task: the General Problem Solver (Newell and Simon, 1950s), state-space search, and the first heuristic and graph-search programs (for example, Dijkstra 1959, whose algorithm is UCS) all treated problem solving as search through a state space.

1.4 Problem formulation

Component	Definition
States	The robot's location: a node of the graph G = (V, E), n =
Initial state	The pharmacy node s
Actions	ACTIONS(u) = {move to v : (u, v) in E}; branching factor b = average degree
Transition model	RESULT(u, move-to-v) = v
Goal test	u = requested ward node g
Path cost	Sum of edge travel times; every step cost >= epsilon = 1 s (weights are integers 1..10)

1.5 Preprocessing

- **Adjacency lists** built from the edge list in Θ(V + E) time and Θ(V + E) space, so that ACTIONS(u) costs O(deg u) instead of O(E).
- **Explored set** (hash set / dictionary of parent pointers) for BFS and UCS: Θ(1) expected duplicate check, up to Θ(V) space. It is created lazily (dictionary), so a short search does not pay Θ(V) to initialise it.
- IDS uses only an **on-path set** (cycle check on the current path), costing O(d) space.

1.6 The three strategies

	A. BFS	B. UCS	C. IDS (recursive)
Idea	Expand level by level with a FIFO queue; the first goal found has the fewest moves	Always expand the cheapest frontier node using a priority queue; the first goal expanded has least travel time	Repeat a recursive depth-limited DFS with limit 0, 1, 2, ...; DFS memory with BFS completeness
Style	Iterative, queue	Iterative, priority queue	Recursive

Why investigated	Fastest simple baseline when all moves are equal; shows the memory limit	Edges have different travel times, so the fewest-moves route is not the fastest	Needs almost no memory, the only strategy that may remain feasible at 10^6 states

A. BFS

```
BFS(adj, s, g):
  if s = g: return [s]
  queue <- [s];  parent <- {s: none}          // explored set
  while queue not empty:
    u <- dequeue(queue)                        // expansion
    for (v, w) in adj[u]:
      if v not in parent:
        parent[v] <- (u, w)
        if v = g: return path(parent, g)       // goal test at generation
        enqueue(queue, v)
  return FAILURE
```

B. Uniform-cost search

```
UCS(adj, s, g):
  heap <- [(0, s)];  best <- {s: 0};  parent <- {s: none};  done <- {}
  while heap not empty:
    (c, u) <- pop_min(heap)
    if u in done: continue                     // stale entry
    if u = g: return path(parent, g), c        // goal test at expansion
    add u to done
    for (v, w) in adj[u]:
      if v not in done and c + w < best.get(v, infinity):
        best[v] <- c + w;  parent[v] <- (u, w);  push(heap, (c + w, v))
  return FAILURE
```

C. Iterative deepening with recursive depth-limited DFS

```
IDS(adj, s, g):
  for limit = 0, 1, 2, ...:
    r <- DLS(s, limit)
    if r is a path: return r
    if r = FAIL: return FAILURE                // no cutoff happened: whole component
searched

DLS(u, limit):                                 // on_path = nodes of current path
  if u = g: return path
  if limit = 0: return CUTOFF if u has a neighbour not on path else FAIL
  add u to on_path;  cut <- false
  for (v, w) in adj[u]:
    if v in on_path: continue                  // avoid cycles
    r <- DLS(v, limit - 1)
    if r is a path: return r
    if r = CUTOFF: cut <- true
  remove u from on_path
  return CUTOFF if cut else FAIL
```

Difference between the strategies: BFS orders by depth (moves), UCS by path cost g(n), IDS by depth with a different memory discipline (path only). Only IDS is recursive; BFS and UCS are iterative.

Task 2 – Basic Operation

For all three algorithms the dominant operation is **node expansion with successor generation** (the loop that examines each neighbour of a node), but what it costs and how often it runs differs.

Algorithm	Basic operation	Where it occurs	Why it is appropriate	How its count grows with state-space size
BFS	Generating a successor: the duplicate test + parent write + enqueue inside the neighbour loop (executed once per generated state)	Body of `for (v, w) in adj[u]` after each dequeue	Every other step (dequeue, goal test) happens at most once per generated state, so it is proportional to the generation count; each costs Θ(1) expected	About b generations per expanded node; expansions grow like b^(d-1) until the whole component is exhausted, then Θ(V + E)
UCS	Heap push/pop of a frontier entry, triggered by a successor generation (relaxation)	Neighbour loop (push) and `pop_min` (pop)	Each costs Θ(log F), so it dominates the constant-time duplicate checks	Grows with the number of nodes whose cost is at most C*, i.e. b^(1+ C*/ε); cheap edges inflate it, and the log factor grows with the frontier
IDS	A recursive call `DLS(v, limit-1)` (one node generation)	Neighbour loop inside `DLS`	One call = goal test + cycle check + loop setup; all other work is proportional to the number of calls	Calls are repeated in every iteration: a node at depth i is generated (d - i + 1) times; there is no global explored set, so a state reached by several paths is generated once per path

The number of basic operations is therefore: BFS = generated nodes (goal test on generation), UCS = heap operations ≈ 2 × generated nodes, IDS = total recursive calls summed over all iterations. These are the counters recorded in Task 9.

Task 3 – Time and Space Complexity (derivations)

Notation. b = branching factor (average successors of a node), d = depth of the shallowest goal (in moves), m = maximum depth, C* = optimal cost, ε = minimum step cost (here 1 s), V = number of states, E = number of edges. For the preprocessing and graph-level bounds, V and E also matter because the explored set caps the search at V states.

3.1 BFS (goal test on generation)

Level i of the search tree holds b^i nodes. Before the goal at depth d is generated, every node of levels 0..d-1 is expanded and (in the worst case) all of level d is generated:

$$N_{gen}(BFS) = 1 + b + b^2 + \dots + b^d = \frac{b^{d+1}-1}{b-1} = O(b^d)$$

$$N_{exp}(BFS) = \sum_{i=0}^{d-1} b^i = \frac{b^d-1}{b-1}$$

Space is the frontier plus the explored set. The frontier holds the whole last level, b^d nodes, and the explored set holds all generated nodes, so space = Θ(b^d) = O(b^d). With duplicate detection each state is generated at most once per incoming edge, giving the graph bound **time O(V + E), space O(V)**, whichever is smaller than b^d.

3.2 UCS

UCS expands every node with path cost less than C*. A path of cost less than C* has at most C^*/ε edges, so the depth of the search is at most $1 + C^*/\varepsilon$ (the +1 comes from generating the goal's neighbours before testing it):

$$N_{gen}(UCS) \le \sum_{i=0}^{1+\lfloor C^*/\epsilon \rfloor} b^i = O\left(b^{\,1+\lfloor C^*/\epsilon \rfloor}\right)$$

Each generation costs a heap push of Θ(log F) with F the frontier size, so time = $O(b^{1+ C^*/\varepsilon } \cdot \log F)$, space $O(b^{1+ C^*/\varepsilon })$. On a graph: **time O(E log V), space O(V)**. When all edges cost the same, $C^*/\varepsilon = d$ and UCS behaves like BFS with a log factor and one extra level.

3.3 IDS (recursive depth-limited DFS)

Iteration with limit l generates the whole tree down to level l, which has (b^(l+1) - 1)/(b - 1) nodes. Therefore the node at level i is generated in each of the d - i + 1 iterations l = i, ..., d:

$$N_{gen}(IDS)=\sum_{i=0}^{d}(d+1-i)\,b^i = \frac{b^{d+2}-(d+2)\,b+(d+1)}{(b-1)^2} = O(b^d)$$

Check with d = 1: 2 + b, and the formula gives (b^3 - 3b + 2)/(b-1)^2 = b + 2. Compared with BFS, (b^(d+2)/(b-1)^2) / (b^(d+1)/(b-1)) = b/(b-1), so the repetition overhead is only a factor b/(b-1) (1.5 for b = 3). Space is the recursion stack plus on-path set: **O(d)** (O(bd) if all pending siblings were stored). Without a global explored set, repeated states are re-generated once per distinct path, so on a graph with cycles the count is bounded by the number of distinct simple paths of length at most d, not by V.

3.4 Preprocessing and overall cost

Algorithm	Preprocessing	Search	Total	Memory total
BFS	Adjacency build Θ(V + E)	O(min(b^d, V + E))	Θ(V + E) in the worst case	Θ(V + E) (graph) + O(V)
UCS	Adjacency build Θ(V + E)	O(min(b^(1+C*/ε), E) log V)	Θ(V + E) + O(E log V)	Θ(V + E) + O(V)
IDS	Adjacency build Θ(V + E)	O(b^d) calls (can exceed V + E)	Θ(V + E) + O(b^d)	Θ(V + E) (graph) + O(d)

The adjacency lists are built once and are common to the three algorithms, so for small d (goal near the pharmacy) the preprocessing dominates IDS, whereas for large d the search term dominates. The explored set is created lazily (a dictionary), so it costs nothing before the first insertion.

Task 4 – Best, Average and Worst Case

The cases depend on where the goal lies in the order in which the algorithm visits nodes, and on the edge costs. Counts below are nodes generated, tree model with branching factor b.

BFS

- **Best case:** s = g (1 node generated, 0 expanded), or the goal is the first successor of the pharmacy (2 nodes generated). The goal is found before any other node is expanded, so the cost is Θ(1).
- **Average case:** the goal is equally likely to be any of the b^d nodes at depth d, so on average half of level d is generated before it is reached: about (b^d - 1)/(b - 1) + b^d/2 = Θ(b^d). Level order means the order inside a level, which depends on neighbour ordering (tie-breaking), changes only the constant.
- **Worst case:** the goal is the last node generated on level d, so all of levels 0..d are generated: (b^(d+1) - 1)/(b - 1). If the goal is unreachable, BFS exhausts the whole connected component: Θ(V + E). The explored set stops it from looping.

UCS

- **Best case:** s = g (1 node), or the goal is reached by the cheapest edge out of the pharmacy: the goal is popped after expanding only the nodes cheaper than it, here Θ(b).
- **Average case:** the nodes generated depend on how many nodes have path cost below C*; with random weights the cost-ordered frontier is similar to BFS levels but wider because paths with many cheap edges (cost 1 to 3 s) stay below C* longer.
- **Worst case:** the cheap-edge detour. If the optimal route has large C* but many nodes lie within cost C* (long chains of ε-cost edges), UCS generates all b^(1+ C*/ε) nodes, more than BFS even when the goal has small depth d, because it must be sure no cheaper route exists. An unreachable goal again exhausts the component: Θ(E log V).

IDS

- **Best case:** the goal is at depth d and is the first branch explored. Iterations 0..d-1 still run in full (the goal is only found when the limit reaches d), so the cost is Σ(l=0..d-1) (b^(l+1) - 1)/(b - 1) + (d + 1) ≈ b^(d+1)/(b - 1)^2 = Θ(b^d). Even the best case is exponential in d, unlike BFS, whose best case is constant when s = g or the goal is adjacent. If s = g, IDS stops at iteration 0 (1 node).
- **Average case:** the last iteration generates about half of its b^d nodes on average before it meets the goal, so the total is ≈ b^(d+1)/(b - 1)^2 + b^d/2.
- **Worst case:** the goal is last in DFS order at depth d: total Σ(i=0..d) (d + 1 - i) b^i. For a goal at the maximum depth m, or a graph with cycles and no explored set, the repeated states multiply the count: the number of simple paths of length at most d. If the goal is **unreachable** and the component has cycles, no iteration returns FAIL, so IDS never stops on its own (this is why a

generation budget is used in the implementation; it cannot prove unreachability, unlike BFS and UCS).

In short: BFS and UCS have a cheap best case and are bounded by the component size in the worst case; IDS has the same order b^d but pays a repetition factor and has no bound on unreachable goals.

Task 5 – Asymptotic Analysis

Algorithm	Time	Memory	Best case (Ω)
BFS	Θ(b^d), and O(V + E) on the graph	Θ(b^d), O(V) on the graph	Ω(1)
UCS	O(b^(1+ C*/ε)), and O(E log V) on the graph	O(b^(1+ C*/ε)), O(V)	Ω(1)
IDS	Θ(b^d) in a tree; in a graph with cycles O(number of simple paths of length ≤ d)	Θ(d)	Ω(b^d) (levels up to d-1 are always searched)

Growth in the hospital. The measured goal depth grows from 5 to 10 moves while n grows by 1000, so b ≈ (1000)^(1/5) ≈ 4 and d ≈ log_b n. Then b^d = Θ(n): the search time of all three algorithms grows roughly linearly in the number of states, and the time to answer a request is bounded below by the cost of reading the part of the map around the pharmacy. The difference is in memory. BFS and UCS keep a frontier of Θ(b^d) = Θ(n) entries (BFS frontier: 272 → 251,000 entries, ×920 for n ×1000), whereas IDS keeps only Θ(d) = Θ(log n) frames (6 → 11, ×1.8).

Which resource limits first. For BFS and UCS **memory** becomes the limit first: the frontier is about a quarter to a half of the state count (251,000 of 1,000,000; 37,000 of 100,000), so each tenfold growth of the hospital multiplies the working memory by ten, which hurts a robot with a small embedded computer long before the CPU time (0.6 s for BFS at 10^6 states) does. For IDS memory is never the issue; **time** limits first: the repeated states in a graph with cycles make the number of generated nodes grow faster than the state count, and it cannot decide that a ward is unreachable. UCS adds a log F factor to the time, so it scales worst (7.5 s at 10^6 states against 0.6 s for BFS).

Meaning for the robot: up to about 10^5 states (0.1 to 0.6 s per request) any of the three is practical; at 10^6 states and beyond the plain uninformed searches need an informed or bidirectional search, a precomputed distance table, or a hierarchical map (wings, floors), so that the search touches a small part of the graph.

Tasks 6 and 7 – Recurrence for the Recursive Algorithm and its Solution

Task 6: formulating the recurrence

The recursive algorithm is `DLS(u, l)`, where l is the **remaining depth** (the depth limit minus the current depth).

Component	Identification from the algorithm

Size of the original problem	l (remaining depth); the top-level call is DLS(s, L)
Size of each subproblem	l - 1: each neighbour is searched with a limit one smaller (the problem shrinks by subtraction, not by division)
Number of recursive calls	One per neighbour not on the path: at most b
Work outside the calls	A constant c: goal test, on-path membership test, add/remove on the path, loop control per child
Base case	l = 0: goal test and the cutoff check, constant cost $c_o = \Theta(1)$ (no further recursion)

Let T(l) be the number of operations (calls and the work attached to them) of DLS with remaining depth l when the goal is not found in that subtree (worst case):

$$T(l) = b\,T(l-1) + c, \quad T(0) = c_0$$

Space has its own recurrence S(l) = S(l-1) + 1, S(0) = 1, so S(l) = l + 1 = Θ(l): one stack frame per level.

Task 7: solving the recurrence

The master theorem does not apply because the problem size is reduced by subtraction (l - 1) and not divided by a constant, so I use the **iteration (back-substitution) method** and verify the result with induction.

$$T(l) = bT(l-1)+c = b\,[bT(l-2)+c]+c = b^2T(l-2)+c(b+1)$$

$$\;\;\vdots\;\; = b^{k}T(l-k) + c\,(b^{k-1}+\dots+b+1)$$

Setting k = l and using $T(0) = c_o$:

$$T(l) = c_0\,b^{l} + c\,\frac{b^{l}-1}{b-1} = \Big(c_0+\frac{c}{b-1}\Big)b^{l}-\frac{c}{b-1} = \Theta(b^{l})$$

Verification by induction. Claim T(l) = A·b^l - c/(b - 1) with $A = c_o + c/(b - 1)$. For l = 0: $A - c/(b - 1) = c_o$. If it holds for l - 1, then T(l) = b[A·b^(l-1) - c/(b - 1)] + c = A·b^l - bc/(b - 1) + c = A·b^l - c/(b - 1) .

Interpretation. The search tree has branching b and depth l, so the work is proportional to its number of nodes, Θ(b^l). Increasing the depth by one multiplies the cost by b, and increasing b by one multiplies it by ((b + 1)/b)^l, which is exponential in depth. With l = d the time is Θ(b^d), with l = m (a deep goal or no goal) it is Θ(b^m), while the memory stays Θ(l).

Extension to iterative deepening. IDS calls DLS with limit L = 0, 1, ..., d:

$$T_{IDS}(d)=\sum_{L=0}^{d} T(L) = A\,\frac{b^{d+1}-1}{b-1}-\frac{c\,(d+1)}{b-1} = \Theta(b^{d})$$

The last iteration alone is A·b^d - c/(b - 1), so IDS costs about b/(b - 1) times the final iteration (1.5× for b = 3, 1.33× for b = 4): repeating the shallow levels is cheap because the last level dominates. Memory is Θ(d), the maximum of S(L) over the iterations. Thus IDS has the time order of BFS and the memory order of DFS, as long as repeated states are not significant (see Task 10 for what happens on a graph with cycles).

Tasks 8 and 9 – Implementation, Testing and Experimental Performance

All 60 tests passed (3 algorithms × 4 sizes × 5 cases): every returned route is a valid path from start to goal, UCS cost equals SciPy's Dijkstra optimum in every reachable case, and BFS and IDS find the same minimum number of moves. Code is in the Appendix and in the delivered files `hospital_search.py`, `run_experiments.py`, `belief_search.py`, `make_plots.py`.

Test set-up (reproducible)

Item	Value
Graph	Random sparse hospital graph, n = 1,000 / 10,000 / 100,000 / 1,000,000 states, about 2n edges (average degree ≈ 4), edge weights 1 to 10 s; the last 5 nodes form a sealed ward (unreachable)
Seeds	Graph seed = 1000 + n; start/goal pair seed = 2026 + n; the three algorithms use the same graph and the same pairs
Test cases per size	3 random reachable pairs, 1 case with start = goal, 1 case with an unreachable goal
Timing	Mean of 5 runs per case (perf_counter, Python 3.13, one core); counters are deterministic
IDS budget	2,000,000 generated nodes, then stop and report "budget exceeded"
Command	`python3 run_experiments.py 1000,10000,100000 res_small.json` and `python3 run_experiments.py 1000000 res_1m.json`, then `python3 make_plots.py`

Results for reachable goals (mean of the 3 pairs)

n	Algorithm	Generated	Expanded	Max frontier / stack	Depth	Path cost (s)	Time (s)
1,000	BFS	480	221	272	5.0	34.0	0.0002
1,000	UCS	846	576	384	5.3	24.3	0.0022
1,000	IDS	1,548	383	6	5.0	34.0	0.0018
10,000	BFS	8,458	5,646	4,029	8.3	48.3	0.0101
10,000	UCS	11,880	7,940	5,282	11.3	37.7	0.0252
10,000	IDS	62,950	15,760	9	8.3	48.3	0.0668
100,000	BFS	70,660	40,200	36,690	9.0	52.7	0.108
100,000	UCS	103,300	49,890	52,800	12.0	36.3	0.301

100,000	IDS	414,800	103,800	10	9.0	52.7	0.592
1,000,000	BFS	388,100	137,100	251,000	10.0	53.0	0.591
1,000,000	UCS	1,205,000	805,000	527,200	12.3	46.3	7.49
1,000,000	IDS	847,700	212,300	11	10.0	53.0	1.50

Optimality of the path cost: UCS was optimal in 12 of 12 reachable cases. BFS and IDS (which minimise the number of moves, not the travel time) were optimal in only 2 of 12 cases, with mean costs 14 to 45 percent higher than UCS.

Unreachable goal and start = goal

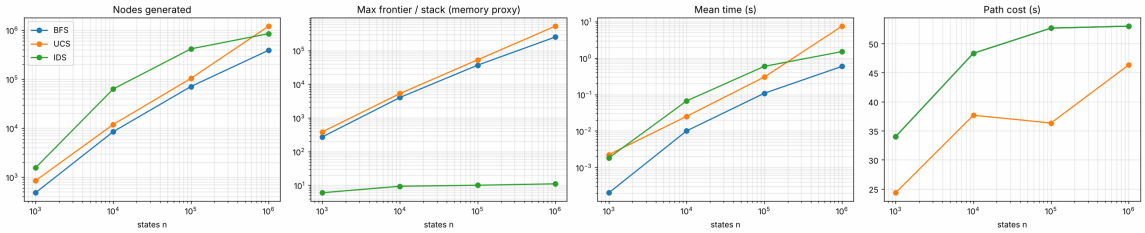
n	BFS generated (time)	UCS generated (time)	IDS generated (time)
1,000	995 (1 ms)	1,230 (2 ms)	2,000,001, budget exceeded (1.26 s)
10,000	9,995 (16 ms)	12,342 (29 ms)	2,000,001, budget exceeded (2.19 s)
100,000	99,995 (0.24 s)	123,791 (0.52 s)	2,000,001, budget exceeded (2.96 s)
1,000,000	999,995 (3.22 s)	1,236,514 (8.68 s)	2,000,001, budget exceeded (3.54 s)

BFS and UCS prove the goal is unreachable by exhausting the component (every state generated once). IDS never terminates on its own because the graph has cycles. For start = goal all three algorithms return a path of length 0 after generating 1 node.

Preprocessing (adjacency lists, mean of 5 builds)

n	1,000	10,000	100,000	1,000,000
Time (s)	0.000	0.012	0.198	2.67

Graphs (log scale)



Tasks 10 and 11 – Theory versus Experiment, and the Final Recommendation

Task 10: comparison

Quantity	Theory	Observed (n = 10^3 →	Agreement and reason

		10^6)	
BFS nodes generated	Θ(b^d) with b^d ≈ n, bounded by V + E	Log-log slope 0.97	Agrees. Absolute values (388,000 at 10^6) are below the worst-case (b^(d+1) − 1)/(b − 1) ≈ 1.4 million because the average goal is found before the last level is finished, and duplicates are removed
BFS frontier	Θ(b^d)	Slope 0.99; frontier ≈ 25 to 40 percent of n	Agrees: the frontier grows in proportion to n
UCS nodes generated	O(b^(1+ C*/ε)), at most E	Slope 1.05; 1.8 to 3.1 times BFS	Order agrees, bound is loose: the formula with C*/ε ≈ 46 is astronomically larger than n, so the graph size (not C*/ε) is the real cap. UCS expanded 805,000 of the 1,000,000 states at n = 10^6, nearly the whole graph
UCS time	O(E log V)	Slope 1.17; time per generated node 6.2 µs at 10^6 against 1.5 µs for BFS	Heap operations (log F) plus the larger dictionaries and cache misses; the 25-fold jump from 10^5 to 10^6 is more than the 12-fold growth in nodes
IDS frontier (stack)	Θ(d)	Slope 0.09; 6 → 11 frames	Agrees closely: stack = d + 1
IDS nodes generated	b/(b − 1) × BFS ≈ 1.33 to 1.5 × BFS in a tree	2.2 to 7.4 × BFS	Disagrees by a factor 2 to 5. The tree model ignores cycles: without an explored set IDS regenerates a state once for each distinct path (the graph has about 2n edges, so many alternative paths). Also the cost of a few random pairs varies, only 3 pairs per size
IDS unreachable goal	Never terminates in a graph with cycles	Hit the 2,000,000 budget at all four sizes	Agrees; BFS and UCS correctly stop after V states

The slopes of 0.9 to 1.05 for generated nodes confirm Θ(n) growth of the generation counts; the time slopes (1.0 to 1.2) are slightly higher because of memory effects (dictionary growth, cache and garbage collection in Python), not the algorithm. Other sources of difference: tie-breaking (neighbour order decides which last-level nodes come first, which affects the average case), the random graph structure (small diameter, so d grows only as log n) and Python interpreter overhead.

Which scales better. For time to find a route, BFS (0.59 s at 10^6) beats IDS (1.5 s) and UCS (7.5 s). For memory, IDS scales best (11 frames against 251,000 for BFS and 527,000 for UCS). For route quality, only UCS returns the least travel time.

Task 11: recommended design

Recommendation: UCS (uniform-cost search) on precomputed adjacency lists, with a duplicate-detection dictionary.

Criterion	Evidence
Route quality	The robot's objective is travel time. UCS was optimal in 12 of 12 reachable tests; BFS and IDS were optimal in 2 of 12 and returned routes that were 14 to 45 percent slower on average (for example 53 s against 46 s at 10^6 states)
Correctness	Terminates on unreachable wards after V states (BFS and UCS), whereas IDS cannot decide unreachability in a cyclic hospital graph
Efficiency	Theory O(E log V) time, O(V) space. Observed 0.025 s (10^4), 0.30 s (10^5), 7.5 s (10^6)
Preprocessing	Adjacency build Θ(V + E): 0.012 s, 0.20 s and 2.7 s at 10^4, 10^5 and 10^6; done once per map change, not per request
Trade-offs and limits	UCS is 2.5 to 13 times slower than BFS and needs the largest frontier (527,000 entries at 10^6). At 10^6 states, 7.5 s per request is acceptable on a server but too slow for interactive use on the robot

Why not the alternatives: BFS is the fastest but ignores travel time, so it is correct only if all corridors cost the same; IDS wins on memory but is slower than BFS, repeats states, and fails to terminate for unreachable wards. A hybrid is reasonable: use BFS first as a cheap reachability check (0.6 s at 10^6) and UCS to produce the final route.

Does it stay practical as the hospital grows? Up to about 10^5 states, yes (0.3 s). At 10^6 and beyond, plain UCS is borderline: both time and frontier grow linearly with n, so a further tenfold growth would, by the measured slope, push the time past a minute and the frontier into the millions of entries (extrapolation, not measured). The next step is an informed search (A* with a straight-line travel-time heuristic) or a hierarchical map; this is outside the uninformed strategies covered in this assignment but is the direction the evidence points to. The recommendation rests on my own measurements, not only on the Big-O comparison: BFS has the lowest time and IDS the lowest memory, but UCS is the only one that meets the performance measure of the robot (least travel time).

Task 12 – Day-4 Design Challenge: Localisation Failure

Decision: MODIFY. The original UCS is kept as the final routing step, but the planner is wrapped in a search over belief states (with a local-sensing conditional plan), because the original assumptions no longer hold.

A. Impact analysis

What changed: the start state is no longer known. The robot only knows it is at one of k junctions, so the problem changes from fully observable to partially observable (sensorless when no sensing is used). Two original assumptions break: (1) full observability of the state, and (2) a known initial state s. Everything else (known map, deterministic moves, step costs) stays.

Why knowledge is part of the state: the effect of an action depends on where the robot really is, which it cannot see. The same action "turn left at the next junction" moves different candidates to different places, so the planner must reason about the set of positions consistent with what the robot knows. That set is the **belief state** b V; the result of an action a is b' = {RESULT(u, a) : u in b}, and the goal is b {dock}. A plan is good only if it reaches the dock from every u in the initial belief.

B. Evaluation of the original design

- **Can it still be applied?** Not directly. Running BFS or UCS from a guessed start gives a route valid only for that guess. In the experiment below, a route planned for a randomly guessed candidate reached the dock from the true location in only **44 percent (k = 2), 38 percent (k = 4) and 28 percent (k = 8)** of 200 trials.
- **What a state becomes:** a belief state, a subset of V. The search space is the set of subsets, 2^V in general. Actions are deterministic, so a belief never grows: starting from k candidates, only subsets of size at most k are reachable, which gives at most Σ(j = 1..k) C(V, j) ≈ V^k/k! belief states. For the test world (V = 38): 741 for k = 2, 82,992 for k = 4, 64,869,363 for k = 8 (against 2^38 ≈ 2.7 × 10^11 in total). For a hospital with V = 10^6 and k = 8 the bound is about 10^48/40,320 ≈ 2.5 × 10^43, so naively searching belief states is not acceptable at hospital scale; the number of candidates k must be kept small and sensing must be used to shrink it.

C. Revised strategy

1. **Sensorless version:** BFS over belief states (frozensets of junctions) using the actions N, E, S, W (a move into a wall leaves the robot where it is). The goal is the belief {dock} (the dock absorbs: once docked the robot stays). Time and space O(min(b^L, V^k/k!)) for plan length L, where the branching factor is the number of actions (4), not the graph degree.
2. **With a local sensor** (the robot senses which of the four sides are open after each move): each action leads to one successor belief per distinct observation, i.e. an AND-OR tree. A conditional plan is found by iterative-deepening AND-OR search (the OR nodes choose an action, the AND nodes must be solved for every observation), memoised by (belief, depth). Observations split beliefs, so the sets become smaller and the search space shrinks.
3. **Hybrid for the full hospital:** run the conditional plan only until the belief becomes a single junction, then the problem is fully observable again and the original UCS is run unchanged from that junction. So the original solution is reused, not discarded.

Effect on complexity: the state space changes from V states to up to V^k/k! belief states; the worst-case plan length is at most about k times the diameter (each phase docks one candidate: the dock absorbs).

D. Evidence-based defence (experiment)

World: 7 × 7 floor plan with random walls, seed 7, largest component containing the dock, N = 38 junctions, dock = (0, 0). The k candidate junctions are drawn with seed 42 (nested draws in one random stream). Command: `python3 belief_search.py`.

k	Sensor	Belief states generated	Worst-case plan length (actions)	Success of the original design
2	none	148	7	44%
2	local wall sensor	150	5	
4	none	2,340	8	38%
4	local wall sensor	442	7	
8	none	958,506	17	28%
8	local wall sensor	681	12	

Reading the table: without sensing, the belief states generated grow from 148 to 958,506 as k goes from 2 to 8 (a ×16 and then ×410 rise, close to the bound V^k/k!), and the plan has to be longer (7 → 17 actions) because it must "squeeze" all candidates to the dock. With the local sensor, the generated count stays below 700 for k = 8 (1,400 times fewer) and the plan is shorter in the worst case (12 against 17), because each observation reduces the belief. The count for the sensor case is the number of (belief, depth) successor evaluations of the AND-OR search, which is not the same unit as the distinct beliefs of the sensorless BFS; for k = 2 the two counts are therefore similar (150 against 148) although the sensor already shortens the plan (5 against 7). The decision is thus supported by both the mathematical bound (V^k/k! growth) and the experiment: retaining the original single-state design fails in 56 to 72 percent of trials, and the belief-state version solves every case, becoming practical only with sensing.

Appendix – Code and Proof of Execution

Files (in the delivered zip `hospital_robot_code.zip`): `hospital_search.py` (graph generation, preprocessing, BFS, UCS, IDS), `run_experiments.py` (tests and measurements), `make_plots.py` (tables and graphs), `belief_search.py` (Task 12), `experiment_log.txt` and `belief_output.txt` (raw outputs), `performance.png`.

Preprocessing and BFS

```
def build_adjacency(n, edges):          # Theta(V + E)
    adj = [[] for _ in range(n)]
    for u, v, w in edges:
        adj[u].append((v, w)); adj[v].append((u, w))
    return adj

def bfs(adj, s, g):
    if s == g: return _result(True, [s], 0, 1, 0, 1)
    q = deque([s]); wpar = {s: None}
    gen, exp, front = 1, 0, 1
    while q:
        u = q.popleft(); exp += 1
```

```
        for v, w in adj[u]:
            if v not in wpar:
                wpar[v] = (u, w); gen += 1
                if v == g:
                    path, cost = _trace(wpar, g)
                    return _result(True, path, cost, gen, exp, front)
                q.append(v)
                front = max(front, len(q))
    return _result(False, None, None, gen, exp, front)
```

UCS

```
def ucs(adj, s, g):
    best = {s: 0}; wpar = {s: None}; heap = [(0, s)]; done = set()
    gen, exp, front = 1, 0, 1
    while heap:
        c, u = heapq.heappop(heap)
        if u in done: continue
        if u == g:
            path, cost = _trace(wpar, g)
            return _result(True, path, cost, gen, exp, front)
        done.add(u); exp += 1
        for v, w in adj[u]:
            nc = c + w
            if v not in done and nc < best.get(v, 1 << 60):
                best[v] = nc; wpar[v] = (u, w)
                heapq.heappush(heap, (nc, v)); gen += 1
                front = max(front, len(heap))
    return _result(False, None, None, gen, exp, front)
```

Recursive depth-limited search inside IDS

```
def dls(u, limit):                      # returns 'found' | 'cutoff' | 'fail'
    path.append(u)
    if u == g: return 'found'
    if limit == 0:
        more = any(v not in on_path for v, _ in adj[u]); path.pop()
        return 'cutoff' if more else 'fail'
    st['exp'] += 1; on_path.add(u); cut = False
    for v, _w in adj[u]:
        if v in on_path: continue
        st['gen'] += 1
        if st['gen'] > max_generated: raise _Budget
        r = dls(v, limit - 1)
        if r == 'found': return 'found'
        if r == 'cutoff': cut = True
    on_path.discard(u); path.pop()
    return 'cutoff' if cut else 'fail'
```

The function `ids` calls `dls(s, limit)` for limit = 0, 1, 2, ... and returns on 'found' or 'fail' (or when the budget of 2,000,000 generated nodes is exceeded).

Sample of the raw output (from `experiment_log.txt`, n = 100,000, first reachable pair)

```
UCS reachable s=51832 g=66865 found=True depth=13 cost=36 gen=112,560 exp=60,581
front=53,014 t=364.1ms optimal=True
IDS reachable s=51832 g=66865 found=True depth=10 cost=55 gen=936,160 exp=234,346
front=11 t=1359.9ms optimal=False
```

```
IDS unreachable s=51832 g=99999 found=False gen=2,000,001 exp=500,623 front=11
t=2958.9ms BUDGET
Correctness failures (excluding IDS budget cases): 0
```

Task 12 output (belief_output.txt)

```
world seed=7: N=38 junctions, dock=(0, 0)
 k    sensor  belief states generated   plan length (worst case)  orig. design
success
 2      none                      148                          7  44%
 2      wall                      150                          5
 4      none                    2,340                          8  38%
 4      wall                      442                          7
 8      none                  958,506                         17  28%
 8      wall                      681                         12
Upper bound on belief states of size <= k: {2: 741, 4: 82992, 8: 64869363}
```